

SSL Guardian: Large-Scale SSL Certificate Analytics Platform



BSCS COMPUTER SCIENCE

Ebad Junaid

Roll No: 22046

Session: BSCS 2022–2026

Muhammad Arslan Shafi

Roll No: 22100

Session: BSCS 2022–2026

DEPARTMENT OF COMPUTER SCIENCE
INFORMATION TECHNOLOGY UNIVERSITY
LAHORE, PAKISTAN



SSL Guardian: Large-Scale SSL Certificate Analytics Platform

A final year project submitted in partial fulfillment of the requirements for the
Degree of Bachelor of Science in
Computer Science

Ebad Junaid

Roll No: 22046

Session: BSCS 2022–2026

Muhammad Arslan Shafi

Roll No: 22100

Session: BSCS 2022–2026

Supervisor:

Dr. Muhammad Umar Janjua

Final Year Project Committee:

Mr. Kashif Junaid

Mr. Shoaib Majeed

DECLARATION

This project is a presentation of my original research work. Wherever contributions of others are involved, every effort is made to indicate this clearly, with due reference to the literature, and acknowledgement of collaborative research and discussions. I also declare that this work is the result of my own investigations, except where identified by references and free from plagiarism of the work of others.

Signature:
Ebad Junaid

Signature:
Muhammad Arslan Shafi

Date:

The undersigned hereby certify that they have read and recommend this project entitled **“SSL Guardian: Large-Scale SSL Certificate Analytics Platform”** by **Ebad Junaid** and **Muhammad Arslan Shafi** for the degree of Bachelor of Science in Computer Science.

Dr. Muhammad Umar Janjua, Supervisor

Mr. Kashif Junaid, Final Year Project Committee Member

Mr. Shoaib Majeed, Final Year Project Committee Member

Dr. Ali Ahmed, Head of the Department

ACKNOWLEDGEMENT

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

All praise and gratitude are due to Almighty Allah, whose countless blessings, guidance, and mercy gave us the strength, patience, and determination to successfully complete this Final Year Project. Without His grace, this achievement would not have been possible.

We would like to express our sincere gratitude to our respected Supervisor for their continuous guidance, valuable time, constructive feedback, and unwavering support throughout this project. Their expertise, encouragement, and insightful suggestions played a significant role in helping us overcome challenges and successfully accomplish our objectives.

We are also thankful to the Department of Computer Science and Information Technology University (ITU), Lahore, for providing us with an excellent academic environment, the necessary resources, and continuous support that enabled us to carry out this project successfully.

As team members, we are grateful for the trust, cooperation, and dedication we shared throughout this journey. By supporting one another, sharing responsibilities, and working together through every challenge, we were able to successfully complete this project.

We sincerely thank our families and friends for their unwavering encouragement, trust, and support throughout this journey.

To everyone who contributed, directly or indirectly, to the successful completion of this project, we extend our heartfelt gratitude.

“If you are grateful, I will surely increase you.”

— Surah Ibrahim (14:7)

DEDICATION

We dedicate this work to Almighty Allah, whose countless blessings, mercy, and guidance made this achievement possible.

We lovingly dedicate this project to our beloved parents and families for their unconditional love, prayers, sacrifices, patience, and unwavering support throughout our journey. Their trust and encouragement have been our greatest source of strength.

We also dedicate this work to our teachers, mentors, and everyone who inspired, guided, and believed in us. May this achievement reflect our gratitude for their support and serve as a meaningful milestone in our academic and professional journey.

TABLE OF CONTENTS

List of Tables	iii
List of Figures	iv
List of Abbreviations	v
Abstract	vii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Project Objectives	2
1.4 Project Contributions	3
1.5 Scope and Boundaries	4
1.5.1 Project Scope	4
1.5.2 Project Boundaries	5
1.6 Report Organization	6
2 Technical Background and Related Work	7
2.1 TLS and the X.509 Certificate Ecosystem	7
2.2 Certificate Transparency and Dataset Freshness	8
2.3 Security-Relevant Certificate Characteristics	9
2.4 Related Work and Research Gap	10
3 Requirements and System Analysis	12
3.1 Stakeholders and Use Cases	12
3.2 Functional Requirements	13
3.3 Non-Functional Requirements and Constraints	13
3.4 Technology Stack	15
3.5 System Components and Context	16
4 System Architecture and Data Design	17
4.1 Architectural Overview and Data Flow	17
4.2 Certificate Data and Scope Model	18
4.3 Indexing, Caching, and Request Processing	19
4.4 Deployment and Architectural Boundaries	20
5 Certificate Acquisition and Dataset Construction	21
5.1 Domain Dataset Preparation	21

5.1.1	Pakistan-Focused Domain Extraction	21
5.2	Domain-Based Certificate Crawling	22
5.3	IP-Based Certificate Acquisition	23
5.4	Acquisition Limitations and Ethical Considerations	24
6	Certificate Transparency Renewal and Discovery Pipeline	26
6.1	Continuous CT Observation	26
6.2	The Periodic Maintenance Cycle	27
6.3	Renewal Detection and New Domain Discovery	29
6.4	Dataset Merge and Analytical Refresh	29
6.5	Component Map	30
6.6	Reliability and Correctness Considerations	31
7	Certificate Analytics and Dashboard	33
7.1	Analytics Processing Framework	33
7.2	Certificate and Security Analytics	34
7.2.1	Validity and Geographic Analysis	34
7.2.2	Signature, Hash, and Encryption Analysis	35
7.2.3	SAN Analysis	35
7.2.4	Shared-Key Analysis	35
7.2.5	Trend Analysis	35
7.3	Security-Oriented Risk Assessment	35
7.4	Certificate Authority Analytics and Ranking	36
7.5	API and Interactive Dashboard	37
7.6	Chapter Summary	38
8	Evaluation and Limitations	40
8.1	Evaluation Method and Verification Results	40
8.2	Experimental IP-Crawler Results	41
8.3	Security and Scalability Assessment	41
8.4	Limitations and Threats to Validity	42
9	Conclusion and Future Work	43
9.1	Conclusion	43
9.2	Future Work	43
	References	45

LIST OF TABLES

1.1	Summary of project scope and principal boundaries	6
2.1	Representative Studies Related to SSL/TLS Certificate Analysis	11
3.1	Primary Stakeholders and Their Objectives	12
3.2	Functional Requirements of the Proposed System	13
3.3	Non-Functional Requirements of the Proposed System	14
3.4	Principal Operational Constraints	14
3.5	Technology Stack of the Proposed Platform	15
4.1	Principal certificate data fields	18
6.1	Mapping of pipeline roles to implementing artifacts.	31
7.1	Certificate analytics capabilities	34
7.2	Application-defined certificate risk score	36
7.3	Certificate Authority ranking components	37
8.1	Verification results	40
8.2	Experimental IP-crawler outcome summary	41

LIST OF FIGURES

2.1	Simplified TLS Handshake and Certificate Authentication Process	7
2.2	Simplified Certificate Transparency Ecosystem	8
2.3	Major Security-Relevant Characteristics of an X.509 Certificate	9
3.1	Logical components and system context of the platform	16
4.1	End-to-end Architecture of the SSL/TLS Certificate Analytics Platform . . .	17
4.2	Logical Organization of Certificate and Analytical Data	19
4.3	Scoped dashboard request and response sequence	20
5.1	Domain-based crawler architecture	22
5.2	IP-based certificate acquisition and result-persistence workflow	24
6.1	The two-branch pipeline: CT observations are split into renewal candidates (left, replacing existing records) and discovery candidates (right, inserted as new records), both converging on the updated dataset that feeds the dashboard.	28
7.1	Materialized and live analytics paths	33
7.2	Interactive dashboard component relationships	38

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AIA	Authority Information Access
APNIC	Asia-Pacific Network Information Centre
API	Application Programming Interface
CA	Certificate Authority
ccTLD	Country-Code Top-Level Domain
CIDR	Classless Inter-Domain Routing
CRL	Certificate Revocation List
CSV	Comma-Separated Values
CT	Certificate Transparency
DER	Distinguished Encoding Rules
DNS	Domain Name System
DV	Domain Validation
ECDSA	Elliptic Curve Digital Signature Algorithm
EV	Extended Validation
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
JSON	JavaScript Object Notation
OCSP	Online Certificate Status Protocol
OV	Organization Validation
PEM	Privacy-Enhanced Mail
PKI	Public Key Infrastructure
RSA	Rivest–Shamir–Adleman
SAN	Subject Alternative Name
SCT	Signed Certificate Timestamp
SHA	Secure Hash Algorithm
SNI	Server Name Indication
SPKI	Subject Public Key Info
SSL	Secure Sockets Layer
SWR	Stale-While-Revalidate
TCP	Transmission Control Protocol
TLD	Top-Level Domain

TLS	Transport Layer Security
UI	User Interface
X.509	Public-Key Certificate Standard

ABSTRACT

SSL/TLS certificates secure traffic across millions of websites, yet many use weak algorithms, carry expired validity, or reuse public keys across unrelated domains. Certificate Transparency logs make these certificates publicly observable, but looking at certificates one at a time reveals only isolated details, not the broader patterns needed to assess issuer behaviour, cryptographic practices, or certificate quality. This report presents SSL Guardian, an integrated platform that collects, maintains, analyzes, and presents large-scale certificate data through a single workflow.

The platform acquires certificates from Internet TLS endpoints using a concurrent, domain-based crawler with persistent task state and stale-task recovery, supported by a separate experimental IP-based crawler. A CT-assisted maintenance pipeline observes public logs, separates renewal candidates from previously unseen domains, re-crawls both, and merges the results so that the dataset stays current. Parsed X.509 records are stored in MongoDB and processed into six materialized analytical families—validity, geography, signature and key, Subject Alternative Names, shared keys, and Certificate Authority—complemented by live queries for time-sensitive views. A Django service exposes these results, and a Next.js dashboard provides scope-aware search, filtering, visualization, and certificate-level drill-down across global and country-level scopes.

The platform was evaluated through structural inspection, static analysis, interface traceability, and one IP-crawler experiment comparing acquisition with and without Server Name Indication. The analytical scores, Certificate Authority rankings, and shared-key labels are project-defined indicators for further investigation rather than externally calibrated security ratings or proof of compromise, and integration, concurrency, and performance testing remain future work.

CHAPTER ONE

Introduction

1.1 Background and Motivation

Secure Internet communication is fundamental to the exchange of personal, organizational, and governmental information. Transport Layer Security (TLS), the successor to Secure Sockets Layer (SSL), protects network communication by providing confidentiality, integrity, and endpoint authentication [1]. In HTTPS, these protections rely on digital certificates that authenticate Web servers and support the establishment of encrypted connections.

Within a Public Key Infrastructure (PKI), trusted Certificate Authorities (CAs) issue X.509 certificates that bind identities to public keys. The Internet profile of X.509 defines certificate fields, extensions, certification paths, and revocation-related structures [2]. A deployed certificate may therefore describe its subject, issuer, validity period, public-key algorithm and size, signature algorithm, certificate policies, and Subject Alternative Names (SANs). Collectively, these attributes provide a basis for evaluating issuer activity, cryptographic choices, validity practices, domain coverage, and other characteristics of the certificate ecosystem. As the number of deployed certificates grows, manual inspection becomes increasingly impractical, motivating automated collection, structured storage, and analysis.

Certificate Transparency (CT) improves the visibility of certificate issuance through publicly auditable, append-only logs [3]. These logs provide a valuable source for monitoring issued or observed TLS server certificates; however, their scale and continuous growth make systematic analysis difficult. Manual review cannot efficiently track changes in validity, issuance patterns, or certificate characteristics across extensive and evolving datasets. This creates a need for analytical approaches that can process large certificate collections while retaining the contextual information required for meaningful security assessment.

Large certificate collections also reveal relationships that are difficult to identify through isolated certificate lookup. Repeated certificates and reused RSA public keys have, for example, been observed in large PKI datasets [4]. Such relationships may remain hidden when certificates are examined individually because their significance emerges only after comparing records across domains, issuers, and time periods. Large-scale analysis is therefore important for exposing patterns that warrant further investigation, while ensuring that observations such as key reuse are not treated, by themselves, as proof of private-key compromise.

The motivation for this project arises from the limitations of manual and lookup-oriented certificate inspection. Examining one certificate at a time may answer a narrow operational question, but it does not provide an ecosystem-wide view of issuer activity, validity practices, cryptographic algorithms, domain coverage, geographic patterns, or certificate-quality

concerns. Researchers, system administrators, and cybersecurity professionals require a systematic means of comparing these properties across large datasets so that significant patterns can be distinguished from isolated observations.

Scalable analytics are necessary because the value of certificate data depends not only on its availability but also on the ability to organize, summarize, and interpret it. Raw records and log entries are difficult to assess directly at scale, whereas clear statistical representations and visual summaries can make distributions, trends, and unusual relationships easier to investigate. This need becomes increasingly important as certificate ecosystems continue to change and as security decisions must be informed by current, comprehensible, and appropriately contextualized evidence.

These practical challenges motivated the development of the proposed certificate analytics platform. The objective is to transform extensive and continuously evolving SSL/TLS certificate data into organized, meaningful, and actionable information that supports large-scale security analysis. This naturally leads to the problem addressed in the following section.

1.2 Problem Statement

The widespread adoption of HTTPS and the continuing issuance of SSL/TLS certificates have produced large and continuously changing collections of certificate data, including records made publicly visible through Certificate Transparency. Although this information is valuable for assessing the security posture of Internet services, manually collecting, organizing, and examining it at scale is impractical. Security-relevant conditions—including expired certificates, weak cryptographic choices, configuration problems, and public-key reuse—may also remain difficult to identify when analysis is limited to individual certificates rather than broader patterns across issuers, domains, and time periods.

Certificate lookup and monitoring tools support focused inspection, but individual-record workflows are insufficient for comprehensive, large-scale security analysis. Researchers, system administrators, and cybersecurity professionals require a scalable means of organizing and comparing extensive certificate datasets, transforming raw certificate attributes into interpretable evidence, and presenting significant distributions and relationships clearly. Consequently, there is a need for a unified certificate analytics platform that supports systematic investigation without relying on manual inspection.

1.3 Project Objectives

The overall objective of this project is to develop a scalable SSL/TLS certificate analytics platform that supports the structured collection, maintenance, analysis, and presentation of large certificate datasets. The project seeks to make certificate ecosystem data more accessible for systematic security research, operational monitoring, and informed assessment. Although the platform supports global analysis, it places a special focus on Pakistan’s certificate

landscape: .pk domains were extracted from multiple global datasets and merged into the primary collection, enabling Pakistan-specific country-level analytical scopes alongside the global view.

The specific objectives of the project are as follows:

1. **Acquire and organize structured certificate data.** Collect X.509 certificate records from large domain datasets in a form suitable for consistent analysis.
2. **Maintain an Up-to-Date Certificate Dataset.** Automatically incorporate newly issued and renewed certificates into the analytical dataset to support continuous and up-to-date assessment.
3. **Support Multi-Scope Certificate Analytics.** Enable comparative analysis of SSL/TLS certificate ecosystems at both global and country-specific scopes.
4. **Provide Near Real-Time Analytics.** Ensure that newly collected certificate data is reflected in the analytical results with minimal delay, allowing users to monitor continuously evolving certificate ecosystems.
5. **Analyze significant certificate characteristics.** Examine validity periods, issuers, cryptographic algorithms, key properties, SANs, and geographic distributions at global and selected country scopes.
6. **Identify security-relevant patterns.** Surface expiration, weak cryptographic choices, public-key reuse, certificate-quality concerns, and application-defined risk indicators for further investigation.
7. **Support comparative assessment of Certificate Authorities.** Evaluate CA activity and provide comparative rankings using explicitly defined certificate-based metrics.
8. **Facilitate interactive certificate analysis.** Present interactive dashboards, statistical summaries, visualizations, search, filtering, and detailed certificate information to facilitate efficient exploration and security analysis.

1.4 Project Contributions

The proposed project makes several technical and practical contributions to large-scale SSL/TLS certificate collection, maintenance, analysis, and visualization. The principal contributions of the developed platform are summarized below.

1. **Intelligent certificate crawling framework.** The project develops automated crawlers that retrieve and structure SSL/TLS certificates for domains obtained from Certificate Transparency (CT) logs and other supported domain sources. The framework combines concurrent domain-based and CT-driven crawling with a separate experimental IP-based crawler, while persistent task states and stale-task recovery mechanisms provide operational recovery support with limited manual intervention.
2. **Continuous certificate dataset maintenance.** A Certificate Transparency driven maintenance process identifies certificates associated with newly observed and renewed

domains, incorporates successfully acquired records into the analytical dataset, and supports its continued expansion over time.

3. **Timely analytical updates.** Certificate acquisition and dataset maintenance are integrated with analytical processing so that refreshed results can be made available after data-update cycles without requiring a separate manual analysis workflow.
4. **Global and country-level analytical scoping.** The platform supports consistent exploration of the complete certificate dataset and configured country-level domain scopes, enabling comparative analysis across multiple geographic scopes within a unified framework.
5. **Comprehensive certificate analytics.** The analytical capabilities cover validity periods, issuers, cryptographic algorithms, key properties, validation levels, Subject Alternative Names, and domain-based geographic distributions, providing multiple perspectives on SSL/TLS certificate ecosystems.
6. **Security-oriented certificate assessment.** The system surfaces expired certificates, weak cryptographic choices, public-key reuse, certificate-quality findings, and application-defined risk indicators to support focused security investigation.
7. **Comparative Certificate Authority evaluation.** The platform evaluates Certificate Authorities using explicitly defined, certificate-derived metrics and presents comparative rankings and distribution patterns.
8. **Interactive analysis and visualization.** The web-based interface combines statistical summaries, charts, tables, search, filtering, and detailed certificate views to make large-scale certificate information accessible for systematic exploration.
9. **Unified certificate intelligence workflow.** By integrating acquisition, dataset maintenance, analytical processing, and interactive presentation, the project converts raw certificate records into structured and interpretable information for cybersecurity research, operational monitoring, and informed assessment.

1.5 Scope and Boundaries

This section defines the functional coverage of the developed platform and the limits within which its results should be interpreted. It distinguishes the capabilities addressed by the project from activities and assurances that fall outside its intended purpose.

1.5.1 Project Scope

The project covers the following principal areas:

1. **Certificate acquisition and dataset maintenance.** Automated crawlers collect SSL/TLS certificates for domains obtained from Certificate Transparency logs and other supported domain sources. Newly observed and renewed certificates extend and update the analytical dataset.

2. **Certificate and security analytics.** The platform examines validity, issuers, cryptographic properties, validation levels, Subject Alternative Names, and domain-based geographic distributions. It also identifies expiration, weak cryptographic choices, public-key reuse, certificate-quality findings, and project-defined risk indicators.
3. **Certificate Authority analysis.** Certificate Authorities are compared using certificate-derived distributions, quality indicators, and project-defined ranking metrics.
4. **Global and country-level analysis.** Certificate data can be examined across the complete dataset and configured country-level domain scopes within a consistent analytical framework.
5. **Interactive analysis and visualization.** Dashboards, charts, tables, trends, search, filtering, and detailed certificate views support systematic exploration of analytical results.
6. **Analytical refresh.** Analytical results can be refreshed after dataset updates so that subsequent analysis reflects the latest completed acquisition cycle.

1.5.2 Project Boundaries

The project is subject to the following boundaries:

1. The dataset does not represent every SSL/TLS certificate deployed on the Internet. Results are limited to certificates acquired from the supported sources.
2. Security findings are indicators for further investigation, not proof of compromise or confirmed vulnerability. Penetration testing and exploitation of target systems are outside the project scope.
3. Certificate Authority rankings and risk indicators use project-defined metrics. They are comparative measures rather than official or externally validated security ratings.
4. Country-level analysis is based on configured domain scopes and does not establish server location, subject jurisdiction, or Certificate Authority location.
5. Analytical refresh depends on completion of the relevant collection and update cycle. Instantaneous or strictly real-time updates are not claimed.
6. IP-based certificate acquisition is experimental and remains separate from the primary domain-based analytical dataset.
7. The platform is intended for research and analytical use; it does not replace production-grade certificate management, threat-intelligence, or formal security-audit services.

Table 1.1 summarizes the major areas covered by the project and the principal boundary associated with each area.

Table 1.1: Summary of project scope and principal boundaries

Area	Included scope	Principal boundary
Acquisition	Automated domain-based and CT-guided certificate collection	Coverage is limited to supported sources
Dataset maintenance	Incorporation of newly observed and renewed certificates	Updates depend on completed acquisition cycles
Analytics	Certificate attributes, security indicators, CA comparison, and trends	Findings and metrics require bounded interpretation
Geographic scope	Global and configured country-level domain analysis	Does not establish physical geolocation
Presentation	Interactive summaries, visualizations, search, filtering, and detailed views	Intended for research and analytical use
IP exploration	Experimental IP-based certificate acquisition	Separate from the primary analytical dataset

1.6 Report Organization

Following this introduction, Chapter 2 establishes the technical background and compares the three related-work papers committed with the project. Chapter 3 presents repository-derived requirements, constraints, technology choices, and module decomposition. Chapter 4 describes the component, data, scope, indexing, cache, and request-flow design. Chapter 5 examines domain acquisition, dataset preparation, and the experimental IP crawler. Chapter 6 explains the CT renewal and discovery workflow. Chapter 7 documents materialized and live analytics, the Django API, and the Next.js dashboard. Chapter 8 records verification evidence, security observations, implementation limitations, and threats to validity without introducing unsupported benchmark claims. Chapter 9 concludes the report and proposes future work derived from verified repository gaps.

CHAPTER TWO

Technical Background and Related Work

2.1 TLS and the X.509 Certificate Ecosystem

Transport Layer Security (TLS) is the standard protocol for protecting Internet communication through **confidentiality**, **integrity**, and **authentication**. **HTTPS** applies TLS to web traffic. During the handshake, the client and server negotiate cryptographic parameters, establish shared secrets, authenticate the server, and derive the symmetric session keys used to protect subsequent application data [1].

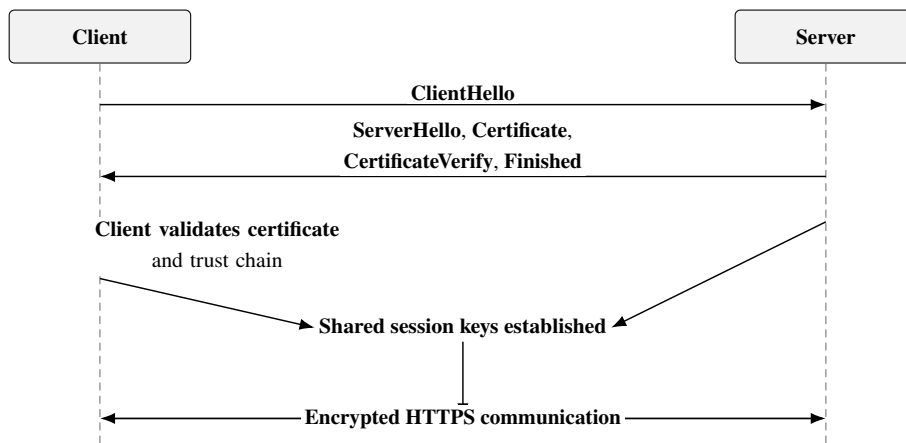


Figure 2.1: Simplified TLS Handshake and Certificate Authentication Process

Public Key Infrastructure (PKI) provides the trust framework used for certificate-based authentication. A **root Certificate Authority (CA)** acts as a trust anchor and delegates issuance through one or more **intermediate CAs**. Together with the server certificate, these certificates form a **trust chain**. A browser validates the signatures, validity periods, identity constraints, and trusted root before accepting the server's identity.

The **X.509** standard defines the structure of public-key certificates. Important fields include the **Serial Number**, **Subject**, **Issuer**, **Validity Period**, **Signature Algorithm**, and **Subject Public Key Info**. Extensions such as **Basic Constraints**, **Key Usage**, **Extended Key Usage**, **Certificate Policies**, and **Subject Alternative Name (SAN)** describe certification authority status, permitted operations, policy context, and authenticated domain names [2]. Collectively, these fields define a certificate's identity, cryptographic properties, and intended use.

Beyond their role in authentication, X.509 certificates provide a rich source of publicly observable security information. Their fields support the study of cryptographic practices, issuer behaviour, deployment patterns, and certificate quality. Comparing these properties

across large datasets enables ecosystem-wide measurements and Internet security research that cannot be achieved through isolated certificate inspection.

Certificate observation differs from **certificate validation**. Browsers trust a connection only after validation, while research measurements may record expired, self-signed, hostname-mismatched, or untrusted certificates. Observation confirms presentation by an endpoint, not acceptance by a standards-compliant client.

2.2 Certificate Transparency and Dataset Freshness

Certificate Transparency (CT) was introduced to reduce the risk that incorrectly or maliciously issued certificates could remain undetected within the **Public Key Infrastructure (PKI)**. It requires certificate issuance to be exposed through publicly accessible, cryptographically verifiable, and **append-only logs**. This public record improves the transparency and accountability of Certificate Authorities by allowing certificate activity and log consistency to be independently examined [3].

Within the CT ecosystem, a **Certificate Authority (CA)** submits an issued certificate or precertificate to a CT log. The log records the entry and returns a **Signed Certificate Timestamp (SCT)** as evidence of promised inclusion. Browsers check for acceptable SCT evidence when evaluating a certificate, while **monitors** and **auditors** observe the public logs for unexpected certificates and inconsistent log behaviour. CT therefore adds public accountability to certificate issuance without replacing normal certificate-chain and hostname validation.

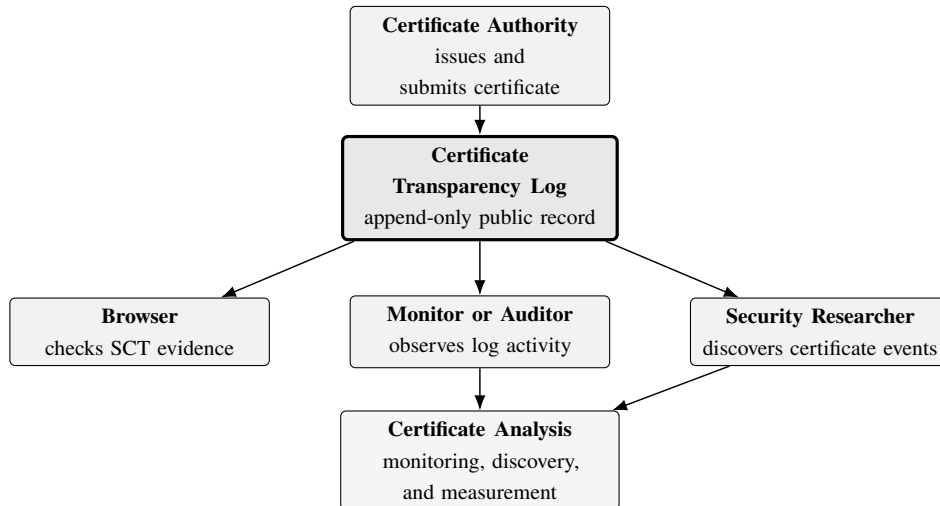


Figure 2.2: Simplified Certificate Transparency Ecosystem

The public visibility provided by CT supports **certificate monitoring**, **certificate discovery**, and the detection of potential **misissuance**. Organizations can search for unexpected certificates associated with their domains, while researchers can examine issuance patterns, CA behaviour, cryptographic characteristics, and changes across large collections. CT logs consequently

provide an important data source for Internet-scale measurement and certificate ecosystem research.

Dataset freshness describes how closely a dataset represents the current state of the environment being studied. Certificate ecosystems change continuously as new certificates are issued, existing certificates expire or are renewed, new domains become active while others are retired or no longer deploy TLS services, and deployed certificates are replaced. A static certificate dataset therefore becomes progressively outdated and may no longer represent current issuance or deployment patterns.

Maintaining a fresh dataset is essential for accurate large-scale analytics, meaningful statistical comparisons, reliable trend analysis, longitudinal security studies, and informed security assessment. Historical snapshots remain valuable for studying change, but current measurements require continuing observation of the evolving certificate ecosystem. Continuously updated datasets are therefore fundamental to meaningful SSL/TLS certificate analytics.

2.3 Security-Relevant Certificate Characteristics

An **X.509 certificate** contains substantially more information than a binding between a domain identity and a public key. Its subject, issuer, validity, cryptographic parameters, extensions, and policy information describe how the certificate was issued and how it is intended to be used [2]. These attributes provide observable evidence about **deployment practices**, **cryptographic strength**, **operational security**, and broader certificate ecosystem behaviour. They are therefore widely used in Internet measurement studies and cybersecurity research.

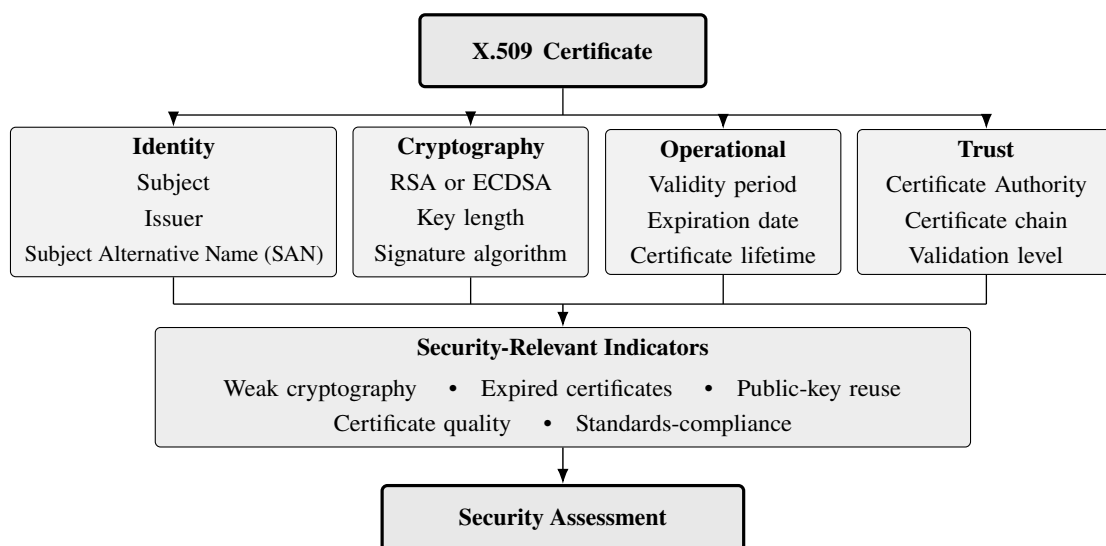


Figure 2.3: Major Security-Relevant Characteristics of an X.509 Certificate

Cryptographic characteristics determine the mechanisms used to authenticate the certificate and protect the associated connection. The **public-key algorithm**, commonly RSA or

ECDSA, identifies the underlying cryptographic approach, while **key length** affects resistance to computational attacks. The **signature algorithm** identifies how the issuer signs the certificate and which hash function supports that signature. Modern algorithms and appropriately sized keys contribute to strong security, whereas obsolete hashes, weak algorithms, or insufficient key sizes may reduce confidence in a deployment. These characteristics must also be interpreted in historical and policy context, because an algorithm accepted when a certificate was issued may later become deprecated as standards and computational capabilities evolve. Algorithm choice and key size therefore indicate whether a deployment follows contemporary security expectations.

Operational characteristics describe certificate lifecycle and trust management. The **validity period** defines when a certificate is intended to be accepted, and the expiration date indicates when it should cease to be used. Expired certificates can interrupt services or reveal weaknesses in renewal and deployment practices, while unusually long validity periods may increase exposure if authorization or key ownership changes [5]. The issuing **Certificate Authority** and certificate chain also provide context about the trust relationship, validation process, and certificate management practices. Issuer patterns across large datasets can additionally reveal trust concentration, differing validation practices, and changes in CA market activity.

Common **security-relevant indicators** include expired certificates, weak cryptographic configurations, repeated public keys, and standards-compliance or certificate-quality findings. Public-key reuse can arise across distinct certificates and has been observed at Internet scale [4]; however, legitimate renewal, cross-signing, and shared infrastructure can produce the same pattern [6]. These characteristics should therefore be treated as signals requiring contextual investigation, not as automatic proof of a vulnerability, successful attack, or private-key compromise. When considered together, identity, cryptographic, operational, and trust attributes provide a broader view of SSL/TLS security posture and support meaningful ecosystem-wide measurement.

2.4 Related Work and Research Gap

SSL/TLS certificates, **Certificate Transparency (CT)**, and Internet-scale certificate ecosystems have been studied extensively over the past decade. Previous research has examined Internet-wide certificate measurement, publicly auditable CT logs, certificate quality, public-key reuse, and certificate lifecycle behaviour. These studies have revealed deployment patterns, issuance errors, shared-key risks, and changes in the certificate ecosystem, thereby substantially improving understanding of the **Web Public Key Infrastructure (Web PKI)**.

Table 2.1: Representative Studies Related to SSL/TLS Certificate Analysis

Study	Primary Focus	Major Contribution	Relevance to This Work
Laurie et al. [3]	Certificate Transparency	Defined publicly auditable logs for observing certificate issuance and identifying suspect activity.	Establishes the basis for CT-assisted discovery and dataset freshness.
Durumeric et al. [7]	Internet-wide HTTPS measurement	Used repeated scans to characterize CA relationships, deployment practices, and configuration problems.	Demonstrates the value of large-scale and time-aware certificate analysis.
Kumar et al. [8]	Certificate quality and CA behaviour	Introduced ZLint to measure certificate conformance with technical standards and policy requirements.	Supports certificate-quality indicators and issuer-oriented assessment.
Junaid and Janjua [6]	Public-key sharing and domain ownership	Analysed shared keys and used domain-ownership relationships to distinguish legitimate and potentially unsafe cases.	Provides context for shared-key detection and cautious interpretation of reuse patterns.
Ma et al. [5]	Certificate lifecycle and staleness	Examined valid but operationally stale certificates and risks associated with prolonged key validity.	Motivates certificate lifetime, renewal, and freshness analysis.

Existing platforms make certificate and Internet observations accessible to researchers and practitioners. **cert.sh** supports certificate search and CT-log exploration [9]; **Censys** combines Internet-wide host discovery with searchable certificate records and security-related metadata [10, 11]; and **Shodan** provides Internet-connected service discovery, TLS certificate filtering, CT lookup, and security monitoring [12, 13]. These platforms have become important resources for certificate discovery, infrastructure measurement, and targeted security investigation.

Previous studies and existing platforms have significantly advanced certificate measurement, CT monitoring, certificate discovery, quality assessment, and Internet-scale security research. However, existing work often concentrates on a specific measurement question, data source, or security characteristic, while relatively few certificate-focused analytical environments integrate **automated certificate acquisition, continuous dataset maintenance, CT-based dataset freshness, comprehensive certificate analytics, Certificate Authority comparison, security-oriented indicators, and interactive visualization**. This systems-integration gap motivates the development of the proposed SSL/TLS certificate analytics platform described in the subsequent chapters.

CHAPTER THREE

Requirements and System Analysis

3.1 Stakeholders and Use Cases

SSL Guardian is intended for four primary stakeholder groups: **cybersecurity researchers**, **security analysts**, **system administrators**, and **students and educators**. These users are interested in understanding SSL/TLS certificate ecosystems, investigating certificate characteristics, identifying security-relevant patterns, and conducting large-scale certificate analysis. The platform supports these objectives by providing an interactive environment for exploring certificate information, analyzing security-relevant characteristics, and performing comparative SSL/TLS certificate analysis.

Table 3.1: Primary Stakeholders and Their Objectives

Stakeholder	Primary Objective
Cybersecurity researchers	Study SSL/TLS certificate ecosystems, deployment trends, and security characteristics.
Security analysts	Investigate expiration, weak cryptographic configurations, public-key reuse, and other security indicators.
System administrators	Examine certificate deployments and lifecycle information to support operational awareness.
Students and educators	Explore SSL/TLS certificates and Public Key Infrastructure concepts for learning and research.

The primary use cases involve searching, filtering, and inspecting certificate records; examining validity, issuer, cryptographic, Subject Alternative Name, and shared-key characteristics; identifying security-related indicators; and comparing Certificate Authorities using certificate-derived measures. Users may also explore trends and visual summaries across global and configured country-level scopes. Together, these activities support research, investigation, operational awareness, and education, and establish the user-oriented expectations formalized in the following functional requirements.

3.2 Functional Requirements

The functional requirements define the principal services and observable capabilities that the developed platform must provide to satisfy stakeholder needs. They describe the major functions expected from the system while remaining independent of the underlying implementation.

Table 3.2: Functional Requirements of the Proposed System

ID	Functional Requirement
FR-01	The system shall accept supported domain datasets and perform automated SSL/TLS certificate acquisition for target domains.
FR-02	The system shall retrieve SSL/TLS certificates presented by target domains and transform them into structured X.509 certificate records.
FR-03	The system shall retain structured certificate information together with the metadata required for analysis, search, and visualization.
FR-04	The system shall process Certificate Transparency observations to distinguish newly observed domains from certificate renewal candidates.
FR-05	The system shall maintain the certificate dataset by incorporating newly acquired and renewed certificates and refreshing analytical results after each update cycle.
FR-06	The system shall generate analytical results for major certificate characteristics, Certificate Authorities, and supported global and country-level scopes.
FR-07	The system shall identify and present expired certificates, weak cryptographic configurations, public-key reuse, and certificate-quality findings as analytical indicators for further investigation.
FR-08	The system shall provide certificate search, filtering, detailed inspection, analytical summaries, and trend visualization capabilities.
FR-09	The system shall present certificate information and analytical results through an interactive web-based dashboard.

3.3 Non-Functional Requirements and Constraints

Non-functional requirements describe the quality attributes expected from the platform and define how well it should perform under its intended operating conditions. They complement the functional requirements by specifying performance and quality expectations without prescribing implementation mechanisms.

Table 3.3: Non-Functional Requirements of the Proposed System

ID	Non-Functional Requirement
NFR-01	The system should support efficient processing of large and continuously growing SSL/TLS certificate datasets.
NFR-02	The system should provide reliable certificate acquisition and analytical operation despite temporary interruptions.
NFR-03	The system should make newly acquired certificate information available for analysis after each completed update cycle.
NFR-04	The system should provide practical response times for interactive search, filtering, and dashboard exploration.
NFR-05	The system should provide an intuitive interface with clear navigation, search, filtering, visualizations, and detailed certificate views.
NFR-06	The system should maintain a modular design that supports future maintenance and extension.
NFR-07	The system should apply analytical metrics, filtering behaviour, and scope definitions consistently throughout the platform.
NFR-08	The system should support responsible certificate analysis by presenting security indicators as evidence for further investigation rather than confirmed compromise.

The developed platform also operates under practical constraints that affect the coverage, timeliness, and interpretation of its analytical results. The principal constraints are summarized below.

Table 3.4: Principal Operational Constraints

Constraint	Description
Dataset coverage	Analytical results are limited to certificates successfully acquired from supported sources and do not represent every SSL/TLS certificate deployed on the Internet.
Update model	Analytical results become available after each completed update cycle rather than through instantaneous refresh.
Geographic scope	Country-level analysis is derived from configured domain scopes and should not be interpreted as verified physical server location.
Analytical interpretation	Security indicators, Certificate Authority rankings, and project-defined metrics support investigation and comparison but do not constitute externally validated security ratings or confirmed compromise.

3.4 Technology Stack

The platform integrates complementary technologies for certificate acquisition, certificate processing, data storage, analytics, server-side services, and interactive visualization. Each technology was selected according to its suitability for a specific role, compatibility with the surrounding components, and support for maintainable development. Table 3.5 summarizes the verified principal technologies used by the platform.

Table 3.5: Technology Stack of the Proposed Platform

Layer	Technology	Purpose / Reason for Selection
Programming language	Python	Supports certificate acquisition, processing, analytics, and server-side services through a mature networking and data-processing ecosystem.
Programming language	TypeScript	Supports maintainable web-interface development through static typing and strong integration with the selected frontend framework.
Backend framework	Django	Provides a mature Python framework for organizing server-side application services and was selected for its reliability and extensibility.
Frontend framework	Next.js	Provides the structure and routing required for the web application while supporting modern React-based development.
User-interface library	React	Enables reusable, state-driven interface components suitable for an interactive analytical platform.
Styling technology	Tailwind CSS	Provides consistent and responsive presentation through a utility-based styling approach.
Database	MongoDB	Stores structured and nested certificate information using a document model suited to heterogeneous X.509 data.
Optional cache	Redis	Provides low-latency reuse of selected analytical responses while allowing the platform to operate when caching is unavailable.
Certificate parsing	ZCertificate	Converts retrieved X.509 certificates into structured machine-readable records for subsequent analysis.
Certificate Transparency	CertStream	Supplies a source of public CT observations and was selected to support continuing certificate visibility.
Visualization	Recharts	Provides reusable chart components that integrate directly with React for interactive analytical presentation.

3.5 System Components and Context

The platform is organized into five cooperating logical components that connect Internet certificate sources with user-facing analytical results. Their principal responsibilities are summarized below.

Certificate Acquisition. Obtains and structures SSL/TLS certificates from supported domain inputs and Internet TLS endpoints.

CT-Assisted Dataset Maintenance. Uses public Certificate Transparency observations to identify newly observed and renewed domain candidates and maintain dataset freshness.

Analytics Processing. Produces global and country-level summaries, trends, Certificate Authority comparisons, and security-relevant indicators.

Application Services. Makes certificate records and analytical results available for search, filtering, detailed inspection, and presentation.

Interactive Dashboard. Enables stakeholders to explore certificate records, summaries, comparisons, tables, and visualizations.

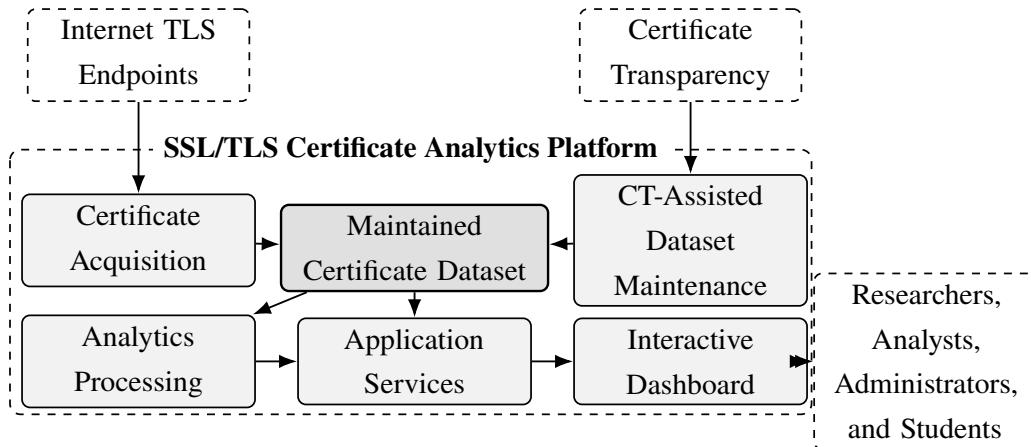


Figure 3.1: Logical components and system context of the platform

Internet TLS endpoints supply certificates for domain-based acquisition, while Certificate Transparency provides observations for dataset maintenance. The collected information supports analytics and application services, and users access the results through the dashboard. The experimental IP-based certificate crawler operates independently and does not contribute to the primary domain-based dashboard dataset. The detailed architecture, data flow, component interactions, and data design are presented in Chapter 4.

CHAPTER FOUR

System Architecture and Data Design

4.1 Architectural Overview and Data Flow

SSL Guardian follows a layered, data-centric architecture that separates data sources, certificate acquisition and maintenance, persistent data, analytical processing, application services, and presentation. The persistent datasets form the principal exchange boundary between these subsystems, allowing each stage to perform a distinct architectural role. Figure 4.1 presents the overall organization and high-level data flow.

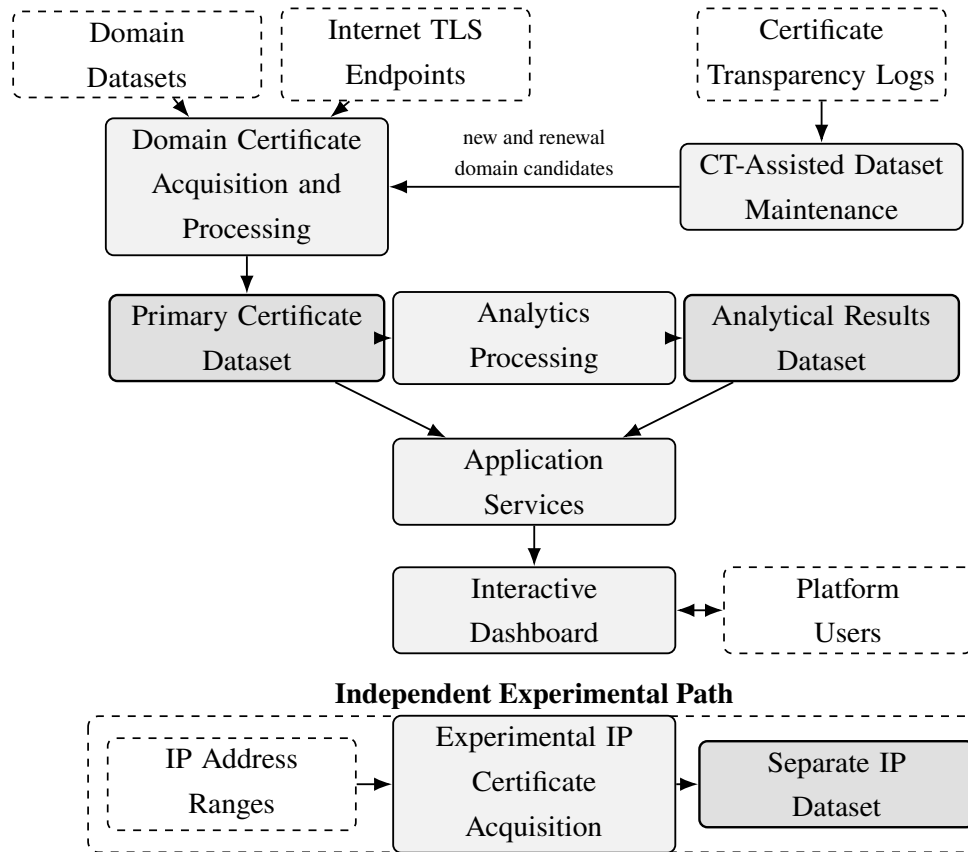


Figure 4.1: End-to-end Architecture of the SSL/TLS Certificate Analytics Platform

The acquisition layer follows two coordinated paths. Domain datasets provide targets for certificate retrieval from Internet TLS endpoints, while Certificate Transparency observations identify new and renewal candidates for the same acquisition capability. Both paths therefore converge on the primary certificate dataset. The experimental IP-based acquisition path remains architecturally separate, and its dataset is not consumed by the primary analytics or dashboard workflow.

The analytical layer transforms primary certificate data into global and country-level analytical results. The service layer accesses both primary records and analytical results, with an optional cache alongside it for selected responses. The presentation layer obtains this information through the application services and exposes it through the interactive dashboard, which forms the system boundary presented to users.

4.2 Certificate Data and Scope Model

Each primary certificate record contains the parsed X.509 object together with attributes introduced during acquisition. Table 4.1 summarizes the principal field groups used by the analytical platform.

Table 4.1: Principal certificate data fields

Field group	Representative content	Analytical purpose
Acquisition context	Domain, derived scope, scan time, and leaf classification	Search, scope filtering, and acquisition tracking
Certificate identity	Serial number and certificate fingerprints	Lookup, deduplication, and certificate correlation
Validity	Start time, end time, and lifetime	Current status, expiry windows, and validity distributions
Issuer and subject	Distinguished names and organization attributes	Identity, issuer, and CA analysis
Public key	Algorithm, key length, and SPKI fingerprint	Cryptographic distributions and shared-key detection
Signature	Signature algorithm and hash information	Algorithm, hash-strength, and compliance analysis
Extensions	SANs, key usages, policies, constraints, AIA, and CRL data	SAN analysis, usage analysis, and CA policy scoring
Certificate-quality findings	ZLint flags and issue categories	Quality grading, risk indicators, and CA comparison

The platform maintains three related logical data groups, illustrated in Figure 4.2. Processing and staging data retains CT observations, acquisition state, temporary results, and recovery metadata; it supports CT-assisted maintenance but is not a mandatory precursor to direct domain acquisition. Primary certificate records provide the detailed analytical source, while analytical results store scope-tagged summaries and shared-key groups derived from that source.

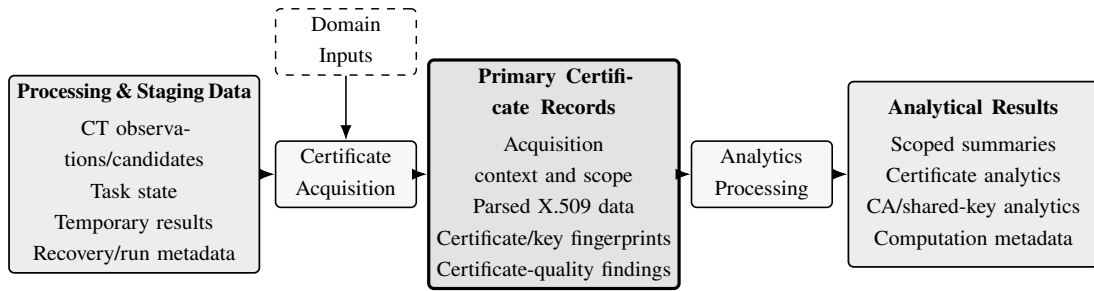


Figure 4.2: Logical Organization of Certificate and Analytical Data

The scope field stores the final label of the acquired domain; it can therefore contain a generic TLD such as `com` or a country-code TLD such as `pk`. Global analysis applies no scope filter to primary records and uses results tagged `all`. Configured country views map each displayed country to its ccTLD and apply that value consistently to primary queries and materialized results. These scopes are domain-based classifications and do not establish server location, issuer jurisdiction, or certificate-subject geography.

4.3 Indexing, Caching, and Request Processing

Indexes support the principal access patterns for scope, domain, certificate identity, public-key fingerprints, validity dates, cryptographic attributes, issuer information, SANs, certificate-quality flags, and leaf classification. Analytical result documents are indexed by scope and by the feature-specific fields required for sorting and filtering. Frequently requested aggregate views are served from scope-tagged, precomputed analytical results, whereas certificate search, selected metrics, and fallback paths use indexed primary records. Pagination and capped sample-identifier lists keep the returned data appropriately bounded.

For applicable request paths, the optional Redis cache stores serialized responses under keys derived from the active scope and request parameters. A cache hit avoids repeated data access; on a miss, or when Redis is unavailable, the service obtains the required information from the primary certificate data or the materialized analytical results and may cache the resulting response. Caching therefore improves repeated access without becoming a requirement for correct operation.

A dashboard request supplies the selected scope and relevant filters, which are applied before the service selects the appropriate data-access path. The result is structured and returned to the dashboard, as summarized in Figure 4.3.

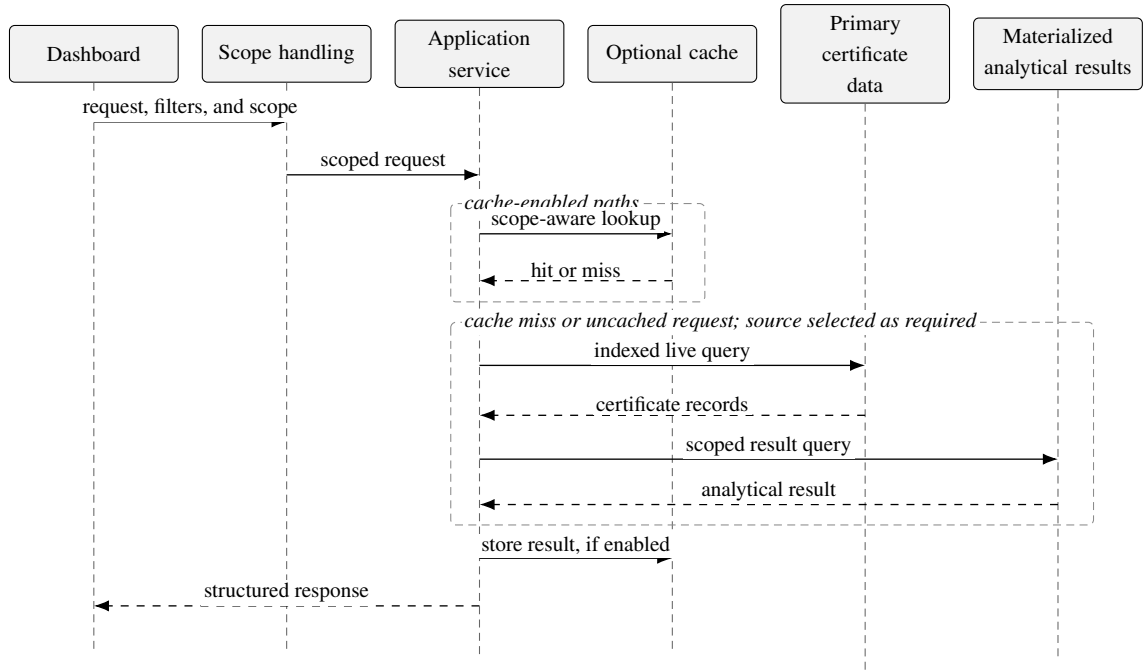


Figure 4.3: Scoped dashboard request and response sequence

4.4 Deployment and Architectural Boundaries

The developed platform is configured as a locally operated research system. MongoDB provides the primary persistent data store, Redis serves as an optional response cache, and Certificate Transparency observation and certificate-parsing processes operate as supporting components. The deployment model does not assume a production cluster or multi-tenant operation.

Several architectural boundaries follow from this model. Scope selection is maintained as shared application state, so strongly isolated concurrent users would require additional request-level separation. Certificate acquisition intentionally bypasses ordinary trust-chain and hostname verification because its purpose is to observe certificates presented by TLS endpoints rather than to establish browser-like trust. Dataset updates and analytical refreshes occur in separate processing stages, with recovery handled within individual stages rather than by one atomic transaction spanning the complete workflow. These limitations and their implications are discussed further in Chapter 8.

CHAPTER FIVE

Certificate Acquisition and Dataset Construction

5.1 Domain Dataset Preparation

The domain-based acquisition workflow receives its initial targets through a configured master CSV dataset, with `global-dataset.csv` serving as the default input for the primary certificate dataset. The master dataset is treated as the operational input without assuming that every contained domain originates from one fixed upstream source. Independent preparation utilities support country-focused lists, Tranco-derived inputs, Rapid7-derived sources, and reconstruction of a domain list from existing certificate records. These utilities provide alternative preparation paths rather than mandatory stages of a single sequential process.

At initialization, domain values from the configured CSV are loaded into the persistent acquisition-task dataset only when that dataset is empty. Each task is created in the pending state with its attempt count initialized and without an assigned worker or heartbeat. A unique domain task key prevents an identical domain value from creating multiple tasks during this loading stage. Once initialized, these records are maintained independently of the original CSV and retain their processing and attempt information across the acquisition run. The persistent task dataset therefore becomes the operational input that the domain-based crawler can subsequently process and monitor.

Domain-derived scope information is not assigned while the input file is prepared or while its tasks are initialized. It is derived from the domain's final label and attached to the certificate record only after successful certificate retrieval and parsing, during record enrichment. The initialized tasks define the starting acquisition population, while confirmed new domains can later extend the master dataset through the Certificate Transparency-assisted discovery workflow described in Chapter 6.

5.1.1 Pakistan-Focused Domain Extraction

In addition to the global domain collection, the project places a special focus on Pakistan's certificate landscape. To build a targeted, localized dataset, `.pk` domains were systematically extracted from four large global sources: the Rapid7 Project Sonar dataset (approximately 15.8 million domains), the Tranco Top Sites list (approximately 4.62 million domains), the Cloudflare and Faculty dataset (approximately 2 million domains), and the Hugging Face dataset (approximately 800,000 domains). Each source was parsed to retain only domains ending in `.pk`, and known Pakistani government and critical infrastructure domains were appended to ensure thorough coverage. The extracted records were combined, deduplicated, and saved into a single file (`pk-domains.csv`) that was merged into the master dataset. This

localized collection enables the dashboard’s country-level analytical scopes and supports Pakistan-specific exploration of certificate characteristics.

5.2 Domain-Based Certificate Crawling

The domain-based crawler uses persistent task records and a configurable pool of worker threads. A worker atomically claims one pending domain, changes its state to processing, and records its worker identity, start time, and heartbeat timestamp. Heartbeat information is refreshed at the principal acquisition stages so that interrupted work can later be distinguished from active work. Figure 5.1 presents the acquisition architecture and its separate recovery path.

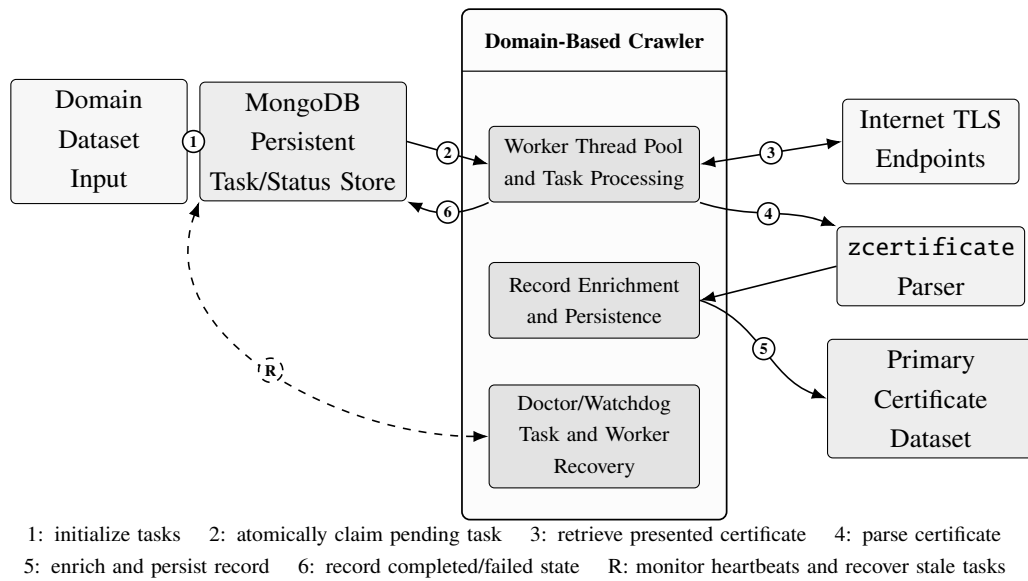


Figure 5.1: Domain-based crawler architecture

For a claimed domain, the worker establishes a TCP connection to port 443 and sends the domain as Server Name Indication during the TLS handshake. Hostname and trust verification are intentionally disabled because acquisition is intended to observe the peer certificate rather than establish a browser-like trusted connection. The presented certificate is retrieved in DER form, converted to PEM, and passed to `zcertificate`, which produces a structured X.509 record. Network communication and parser execution use independently configurable timeouts, both set to ten seconds by default.

After successful parsing, the record is enriched with the source domain, acquisition time, a scope derived from the domain’s final label, and a pragmatic leaf-certificate flag. The flag excludes certificates whose subject and issuer distinguished names are identical or whose basic constraints mark them as a Certificate Authority. The enriched record is inserted into the primary certificate dataset before the task is marked completed. Domain-task uniqueness prevents duplicate task creation during initialization, whereas certificate persistence does not

enforce fingerprint-based uniqueness and therefore does not provide an exactly-once insertion guarantee.

Failures increment the task's attempt count and retain an error description. The retry limit and delays are configurable, although retry processing is disabled by default; a task without an enabled and eligible retry is marked failed. The Doctor/Watchdog identifies processing tasks whose heartbeat has become stale, returns them to the pending state, and can restore missing worker capacity. This recovery path preserves the explicit pending, processing, completed, and failed states without claiming end-to-end exactly-once processing.

5.3 IP-Based Certificate Acquisition

IP-based acquisition provides a complementary method for observing certificates directly from IPv4 endpoints without requiring an associated domain name. Input may consist of individual IPv4 addresses or CIDR prefixes; a supporting utility can derive Pakistan-focused CIDR inputs from APNIC delegated statistics. The acquisition workflow validates the selected input format and expands each CIDR prefix into its usable host addresses before submitting the resulting targets for concurrent processing.

For each acquisition run, the operator selects either a dummy hostname for Server Name Indication (SNI) or a connection without SNI. This choice is significant for virtual-hosted services because an endpoint may return a default certificate, an unrelated certificate, or reject an unrecognized SNI. The resulting observation therefore represents the certificate presented under the selected handshake context rather than a domain-specific assertion.

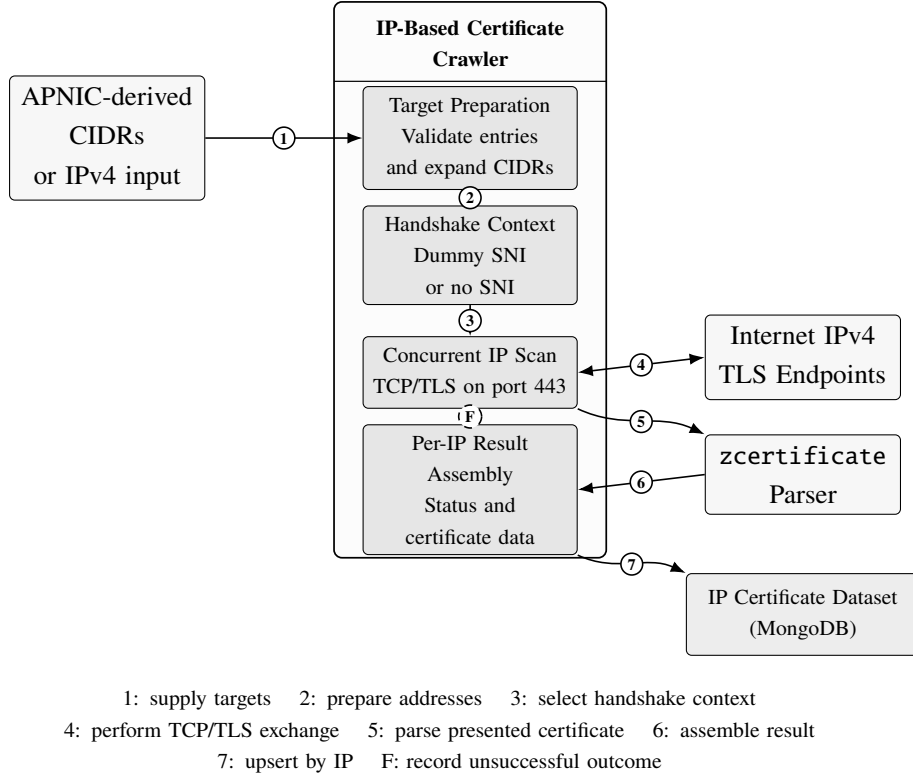


Figure 5.2: IP-based certificate acquisition and result-persistence workflow

As shown in Figure 5.2, each target is first tested for a TCP service on port 443 and then for TLS negotiation. A successful handshake returns the presented certificate, which is converted to PEM, parsed with `zcertificate`, and stored without raw certificate fields. The workflow records differentiated outcomes for connection refusal, TCP or TLS timeout, TLS protocol errors, parsing failure, and successful acquisition. Results are upserted by IP address, so a later scan replaces the prior record for that address.

The IP-based crawler uses a concurrent scan pool and an IP-unique persistence key, but it does not share the persistent task-state, retry, or Doctor/Watchdog model of the domain-based crawler. Its observations are maintained in a dedicated IP-based dataset rather than being merged into the primary domain-based analytics dashboard. This separation preserves the semantics of IP-directed measurement while allowing it to complement domain-based certificate acquisition.

5.4 Acquisition Limitations and Ethical Considerations

Acquisition is bounded by configured inputs and scan-time conditions. DNS failure, unreachable hosts, closed ports, timeouts, TLS errors, parser failure, and transient network conditions can prevent collection. Deployments may also change over time, while IP scanning may return a default virtual-host certificate when SNI does not identify the intended service. Results are therefore point-in-time observations rather than complete Internet TLS coverage.

Certificate collection is distinct from certificate validation. With normal trust and hostname checks disabled, an observed certificate may be expired, self-signed, untrusted, hostname-mismatched, or part of an invalid chain. Its presence records only what the endpoint presented and does not imply acceptance by a standards-compliant client.

The implemented mechanisms perform ordinary TCP/TLS interaction against finite configured targets, use bounded timeouts, and retrieve only publicly presented certificate information; they neither exploit services nor request application data. The software does not centrally enforce rate limits, exclusions, opt-out handling, abuse response, consent, or ethics approval. Responsible deployment must therefore supply appropriate authorization and approval, conservative pacing, target exclusions, monitoring, and retention and disclosure controls. These are governance requirements rather than implemented crawler capabilities.

CHAPTER SIX

Certificate Transparency Renewal and Discovery Pipeline

Certificate Authorities (CAs) participating in the Web Public Key Infrastructure are required by the CA/Browser Forum Baseline Requirements to submit every issued certificate to multiple public Certificate Transparency (CT) logs [3]. These logs maintain append-only, cryptographically verifiable records of all certificate issuance events. The public visibility created by this obligation makes CT logs a valuable observational source for monitoring certificate activity, including detecting newly issued certificates, identifying renewals of existing deployments, and observing changes in the certificate ecosystem over time. The pipeline described in this chapter converts these raw CT observations into a continuously maintained dataset by collecting domain events in real time, periodically separating renewed certificates from those belonging to previously unseen domains, acquiring current endpoint data for both groups, and incorporating the results into the analytical dataset.

6.1 Continuous CT Observation

The Certificate Transparency observation subsystem supplies the domain-level events that drive the maintenance workflow. It comprises three cooperating artifacts: a CT log listener, a consumer process, and a per-log position index.

The CT log listener maintains persistent connections to a configured set of public logs—operated by Google, Cloudflare, Let’s Encrypt, DigiCert, Sectigo, and others. As certificates are submitted to these logs, the listener decodes each issuance event and exposes the observed domain sets through a local WebSocket endpoint, delivering a stream of domain and Subject Alternative Name (SAN) observations extracted from each processed certificate.

A consumer process maintains a persistent WebSocket connection to this endpoint. For each received message, it strips wildcard markers, deduplicates the domain set within that message, and accumulates unique observations in an in-memory buffer. Each buffered entry records the list of observed domains together with a processing flag initially set to `false`. When the buffer reaches a configured threshold, the accumulated records are flushed to MongoDB in a single batch write. A unique index on the `domains` field ensures that repeat observations of the same domain set are silently rejected at the database level without disrupting the ingestion flow.

Two complementary persistence mechanisms support uninterrupted operation across process restarts. The per-log position index records the most recently processed entry position for each monitored CT log, allowing the listener to resume observation from that position rather than reprocessing the complete log history. Separately, the consumer records ingestion

metadata—batch counts, processing timestamps, and run status—in a dedicated MongoDB collection. The processing flag on each buffered domain set serves as an additional tracking mechanism: it is set to `true` when the corresponding set has been considered by the renewal and discovery workflow, preventing the same observation from triggering repeated processing cycles.

This design supports continuous, low-latency observation of CT log activity. On graceful shutdown, any remaining entries in the in-memory buffer are flushed to MongoDB before the process terminates. The buffer is not a durable message queue and does not guarantee exactly-once delivery for every domain observation, but this is acceptable given that the subsequent maintenance cycle processes the collected data in bulk and can tolerate occasional duplicate or missed observations within practical bounds.

6.2 The Periodic Maintenance Cycle

While the observation subsystem runs continuously, the maintenance operations that convert collected observations into dataset updates execute as a periodic, orchestrator-driven cycle. An orchestration component sequences several discrete stages in a defined order, ensuring that each completed cycle produces a coherent, self-consistent update to the certificate dataset and the corresponding analytical results. Figure 6.1 illustrates the overall flow.

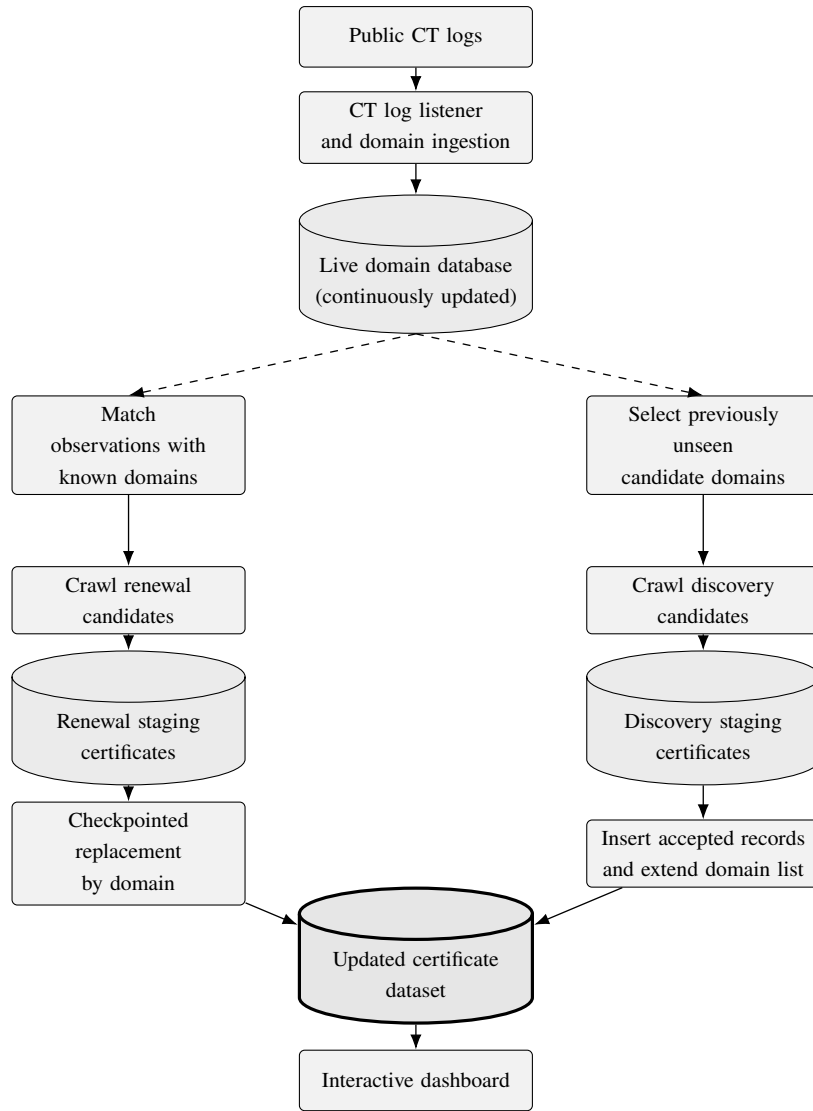


Figure 6.1: The two-branch pipeline: CT observations are split into renewal candidates (left, replacing existing records) and discovery candidates (right, inserted as new records), both converging on the updated dataset that feeds the dashboard.

The cycle begins by temporarily stopping the CT observation subsystem so that the accumulated domain data in the live database remains stable throughout the maintenance run. The orchestrator then verifies that the global domain dataset exists and, if necessary, generates it from the primary MongoDB collection. The core processing stage separates the buffered CT observations into two categories—domains already present in the maintained dataset (renewal candidates) and domains that have not been observed before (discovery candidates)—and processes each category independently through the certificate acquisition workflow described in Chapter 5. After acquisition completes, the newly obtained certificates are merged into the main dataset using different strategies for each category: renewal results replace existing records, while discovery results are inserted as new entries. The analytical results are then recomputed, temporary staging data is removed, and the CT observation subsystem is restarted to resume collection for the next cycle.

6.3 Renewal Detection and New Domain Discovery

Renewal detection determines which of the observed CT domain sets correspond to certificates already represented in the maintained dataset. For each domain in the global domain list, the system generates base-domain and `www` variations and queries the buffered CT observations in the live domain database. A unique index on the stored domain sets allows each observation to be located efficiently by any domain that appears in its SAN list.

When a matching observation is found, only one representative domain is selected for the renewal output, even when multiple domains in the dataset share the same certificate (indicated by identical SAN sets). This prevents redundant crawling of the same certificate endpoint from multiple domain entries. The corresponding CT observation is marked as processed so that subsequent maintenance cycles do not re-examine it. The resulting renewal list therefore contains one entry per renewed certificate rather than one entry per matching domain, reducing the acquisition workload while maintaining full coverage of the observed renewal activity.

The discovery path extends the dataset with certificates for domains that have not been previously observed. It begins by extracting a bounded number of unprocessed domain records from the live domain database, writing them to a temporary input file. The source collection is then dropped, ensuring that the next observation cycle starts with a clean buffer.

The domain-based crawler is launched against this extracted input and runs to completion. Each successfully acquired certificate is stored in a staging database. In the subsequent merge step, the newly acquired certificates are inserted into the main dataset as new records keyed by domain. Because modern certificates commonly cover a large number of Subject Alternative Names (SANs), a single certificate may be acquired independently for several different domains during a single discovery cycle. No fingerprint-based deduplication is applied—each domain receives its own independent record in the dataset, which is the desired behaviour given that the same certificate may serve multiple distinct web properties. The system tracks exactly which domains were successfully inserted and appends only those confirmed domains to the global domain list, maintaining consistency between the MongoDB records and the domain list used for subsequent acquisition cycles.

6.4 Dataset Merge and Analytical Refresh

The renewal merge transfers the certificates acquired during the renewal crawl from the staging database into the main certificate dataset. Each renewal document is written using a replacement operation keyed on the domain field: if a record for that domain already exists in the main collection, it is replaced in its entirety; if no record exists, a new document is inserted. This approach avoids the two-step delete-and-insert pattern, which would create a window during which the dashboard could read an incomplete certificate record.

Progress is tracked through a checkpoint stored in a metadata collection within the main database. The checkpoint records the identifier of the last successfully processed renewal document and is advanced only after each batch write completes. If the merge process is interrupted—by a crash, a timeout, or an explicit stop request—a subsequent run resumes from the stored checkpoint rather than reprocessing the complete renewal dataset. Because each replacement operation is idempotent, rewriting the same document multiple times produces identical results, making partial progress always safe to resume.

After the certificate dataset has been updated, the analytics process verifies the required database indexes and recomputes the six materialized analytical families. Detailed certificate queries and selected time-series views read directly from the main dataset and therefore reflect the current state immediately. Materialized dashboard views, by contrast, serve the latest completed analytical snapshot. The resulting availability is near real time in the sense that it follows the completed maintenance cycle; it is not an instantaneous streaming update driven by individual CT events.

6.5 Component Map

Table 6.1 maps each functional role described in this chapter to the concrete artifact that implements it. Most artifacts reside in the `ct-logs-renewal-pipeline/` directory; the orchestrator additionally invokes the domain-based crawler in `ssl-certificates-crawler/` and the analytical refresh script `run-generic.py` in `dashboard/backend/pre-compute-scripts/`. All stages are coordinated by the orchestrator described in Section 6.2.

Functional role	Artifact	Responsibility
CT log listener	<code>certstream-server-go</code>	Maintains persistent connections to public CT logs and exposes domain events over a local WebSocket
Observation consumer	<code>go-server.py</code>	Subscribes to the listener, buffers observations in memory, and batch-writes unique domain sets to MongoDB
Log position index	<code>ct_index.json</code>	Records the last-seen position per CT log for resumption after a restart
Orchestrator	<code>main.py</code>	Sequences the stages of each maintenance cycle
Master domain list	<code>global-dataset.csv</code>	Authoritative list of known domains, generated by <code>fetch-domains-names.py</code>
Renewal detection	<code>data-renew.py</code>	Matches buffered observations against known domains
Renewal merge	<code>data-renew-merge.py</code>	Performs checkpointed, domain-keyed replacement of renewal certificates into the main dataset
Discovery acquisition	<code>new-data.py</code>	Crawls previously unseen domains and inserts new certificates into the main dataset
Analytical refresh	<code>run-generic.py</code>	Recomputes the required indexes and the six materialized analytical families

Table 6.1: Mapping of pipeline roles to implementing artifacts.

6.6 Reliability and Correctness Considerations

The pipeline incorporates several stage-level recovery mechanisms. CT-log indices in the observation service allow collection to resume from a known position. Persistent processing flags prevent the same domain set from triggering repeated processing cycles. Acquisition task states, batch checkpoints, process-state tracking, and failure logs allow individual stages to be resumed or repeated after interruption without restarting the complete workflow from the beginning. In practice the orchestrator is launched on a fixed schedule through the operating system’s task scheduler (cron on Linux/macOS, Task Scheduler on Windows), with each invocation running its stages to completion before the next cycle begins.

The complete maintenance cycle is not, however, enclosed within a single transaction. CT observation, temporary staging, domain-list updates, certificate merges, analytical recomputation, and subsystem restarts each remain separate operations. The renewal

checkpoint must remain consistent with the lifetime of its staging data—if the staging database is cleared before the checkpoint is advanced, the merge cannot be resumed. Concurrent maintenance cycles also require explicit operational coordination, because overlapping runs would read from and write to the same staging and main datasets without isolation guarantees.

Consequently, the platform provides a practical, automated dataset maintenance process that significantly reduces the manual effort required to keep the certificate collection current. It does not claim global exactly-once processing for individual CT events, instantaneous end-to-end updates in response to every observed issuance, or experimentally measured freshness bounds. These characteristics are appropriate for the intended research and analytical use case, where periodic, coherent snapshots of the certificate ecosystem support meaningful analysis without requiring strict real-time event processing.

CHAPTER SEVEN

Certificate Analytics and Dashboard

This chapter describes how the platform transforms raw certificate data into analytical insights through a combination of pre-computed materialized results and live queries. It presents the six analytical families computed by the processing framework, the security-oriented risk assessment model that ranks individual certificates by observable risk signals, the Certificate Authority ranking formula that compares issuer-level certificate quality, and the interactive dashboard that exposes these capabilities through structured API endpoints and visual pages.

7.1 Analytics Processing Framework

The analytical layer combines materialized result documents with live queries over the certificate dataset. Materialized processing first creates the database indexes required by the analytical workloads and then computes six scope-aware analytical result families, which are stored separately for efficient retrieval. Live queries complement these precomputed results by accessing the main certificate dataset directly whenever current certificate records, detailed views, selected counts, risk candidates, or time-series information are required. Figure 7.1 illustrates the relationship between the materialized analytics pipeline and the live query path.

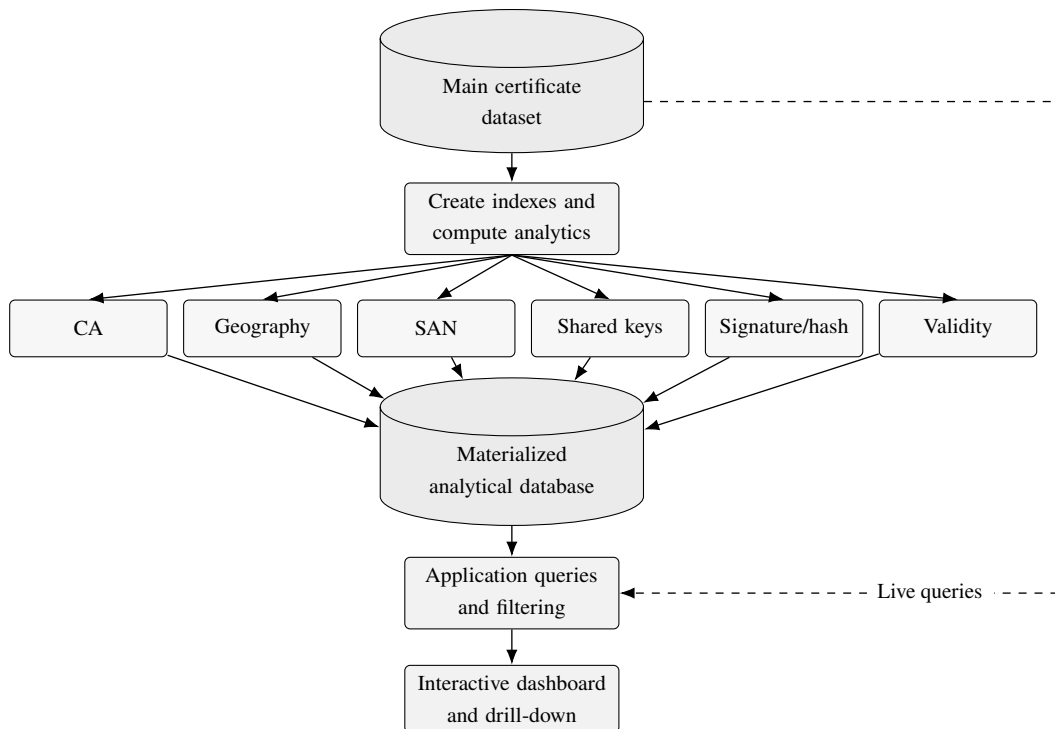


Figure 7.1: Materialized and live analytics paths

The precomputation framework is a single orchestrator script that sequences the creation of database indexes followed by the execution of six independent analytical modules. Each module reads from the main certificate collection, computes its aggregations, and writes the results into a dedicated results database. Because the materialized results are recomputed on a fixed cycle rather than updated incrementally, they represent a coherent point-in-time snapshot of each analytical family. All analytical families support a global view and configured country-level domain scopes. Live queries complement these snapshots by reading directly from the main dataset for views that require current data, such as recent issuance activity and near-expiry windows. Table 7.1 summarizes the principal capabilities.

Table 7.1: Certificate analytics capabilities

Family	Principal analysis	Delivery mode
Validity	Lifetime statistics, expiry status, duration buckets, issuance, and near-expiry windows	Materialized summaries with selected live counts
Geography	Distribution derived from domain suffix and configured scope	Materialized by scope
Signature and key	Signature algorithms, hash families, key algorithms, key sizes, and self-signed records	Materialized summaries with live encryption counts
SAN	SAN totals, count groups, wildcard use, suffix groups, and certificate references	Materialized summaries and drill-down
Shared keys	Distinct certificates grouped by repeated public-key fingerprint	Materialized groups, metadata, and detail views
Certificate Authority	Issuer distributions, validation levels, self-signed counts, and comparative ranking	Materialized by scope

7.2 Certificate and Security Analytics

This section describes the certificate-level analytical families computed by the processing framework.

7.2.1 Validity and Geographic Analysis

Validity processing converts the encoded certificate lifetime into days and calculates average, minimum, maximum, and duration-group statistics. It also supports issuance and expiration timelines together with live 7-, 30-, and 90-day expiry windows. Geographic categories

are derived from the domain suffix through a configured mapping. They describe domain namespace and must not be interpreted as verified physical location.

7.2.2 Signature, Hash, and Encryption Analysis

Certificates are grouped by signature algorithm, hash family, key algorithm, and key size. SHA-1 is treated as deprecated, while MD5 and MD2 are treated as critical hash choices. Live encryption views count selected RSA and ECDSA algorithm-size combinations. The resulting score and strength labels are project-defined analytical indicators rather than standardized security ratings.

7.2.3 SAN Analysis

The SAN family calculates total and average SAN counts, single-name and multi-name distributions, wildcard occurrence, count buckets, and common suffix groups. Bounded certificate references connect aggregate groups to paginated certificate drill-down.

7.2.4 Shared-Key Analysis

Shared-key analysis identifies public keys that appear in more than one distinct certificate. When multiple certificates share the same public-key fingerprint, they are grouped together. Each group records affected certificates, domains, SANs, issuers, key characteristics, and summary counts. Exposure labels use the number of certificates and SANs to support prioritization, but a shared key does not by itself establish private-key compromise or unsafe cross-owner use.

7.2.5 Trend Analysis

Beyond the six pre-computed families, the platform provides live trend analysis that queries the certificate dataset directly. Trend analysis tracks how certificate characteristics evolve over time. Issuance and expiration timelines show the number of certificates created and scheduled to expire in each period. Signature-algorithm adoption charts how usage has shifted, for instance from SHA-1 toward SHA-256. Validation-level trends compare the adoption of Domain Validation (DV), Organization Validation (OV), and Extended Validation (EV) over time. Key-size series plot the distribution of RSA modulus lengths across the dataset. Expiration views group certificates by their notAfter dates and are not predictive models.

7.3 Security-Oriented Risk Assessment

Beyond the aggregate analytics families, the platform maintains a certificate-level risk view that ranks individual certificates by the number and severity of observable security signals.

The view forms a bounded union of certificates matching security-relevant signals, removes duplicate records, applies the score in Table 7.2, and ranks the resulting candidates.

Table 7.2: Application-defined certificate risk score

Signal	Points	Condition
Expired certificate	+30	Certificate status is expired
Shared public key	+30	Public-key fingerprint belongs to a shared-key group
Weak encryption	+20	RSA modulus length is below 2,048 bits
Long validity	+10	Encoded certificate lifetime exceeds 398 days
Certificate-quality findings	+1 to +10	One point per ZLint error and one point per two warnings, capped at ten
Positive characteristic	−5 each	Current validity, strong key, shorter lifetime, and absence of lint findings

The final value is clamped to the interval $[0, 100]$ and classified as **Critical** (85 or above), **High** (70–84), **Medium** (40–69), or **Low** (1–39). Scores of zero are not returned, meaning a certificate must exhibit at least one risk factor to appear in the view. Because candidate queries are bounded and the weights are defined by the application, the result is a prioritization aid rather than proof of vulnerability, compromise, or exhaustive risk coverage.

7.4 Certificate Authority Analytics and Ranking

The Certificate Authority (CA) view aggregates the analytics above to the issuer level. CA analysis groups leaf certificates by issuer organization, validation level, self-signed status, and issuer-related characteristics. Comparative ranking is based on five component groups shown in Table 7.3. Each component is computed as the mean of several sub-metrics, ensuring that no single factor dominates the overall score.

Table 7.3: Certificate Authority ranking components

Component	Principal inputs	Purpose
Core hygiene	Weighted ZLint errors and warnings plus selected critical findings	Measures certificate-quality findings
Cryptographic health	Key length, algorithm family, and public-key reuse	Represents cryptographic and key-management indicators
Operational stability	Certificate authority and operational indicator values	Captures operational characteristics defined by the model
Policy compliance	Extended key usage, policy identifiers, validation level, and name constraints	Represents policy-related certificate attributes
Risk factors	Issuer-country category, authority-access information, and revocation-service presence	Adds contextual risk and revocation indicators

For certificate i , the ranking score is the mean of the five normalized component values:

$$S_i = 100 \times \frac{H_{\text{core}} + H_{\text{crypto}} + H_{\text{operational}} + H_{\text{policy}} + H_{\text{risk}}}{5}. \quad (7.1)$$

The score for a Certificate Authority is obtained by averaging the scores of its included leaf certificates. The output also records the number of certificates contributing to the ranking and the component averages. The formula supports transparent comparison within the platform but has not been externally calibrated as an industry security rating.

7.5 API and Interactive Dashboard

The Django backend exposes a JSON API that serves all dashboard views. Each API endpoint maps to one or more analytical queries, supporting configurable country scope, pagination, and certificate-level filtering by status, issuer, text search, cryptographic attributes, validity windows, SAN characteristics, shared-key membership, and risk signals.

The frontend consumes these endpoints through a typed API client that handles request construction and response parsing. A global SWR provider manages data fetching with automatic deduplication, revalidation, and caching, while React contexts coordinate scope, search, theme, and dashboard state across pages. Figure 7.2 summarises the presentation architecture.

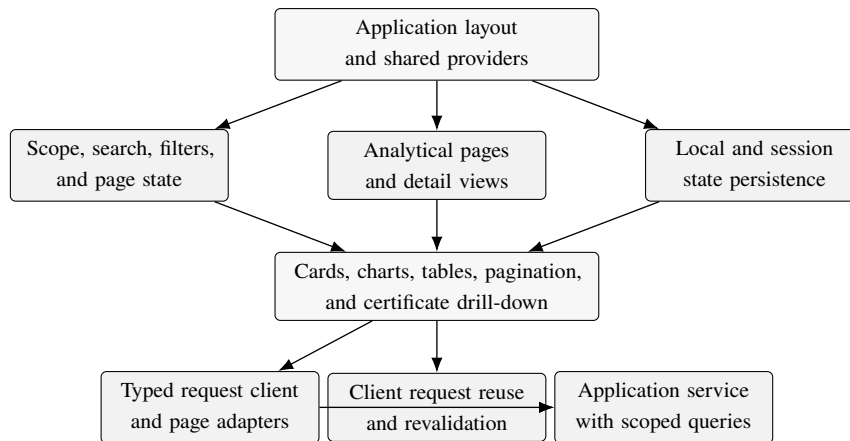


Figure 7.2: Interactive dashboard component relationships

Users navigate the dashboard through a sidebar that lists each analytical family as a dedicated page. The landing page presents a platform overview with total, active, expiring-soon, and expired certificate counts alongside a searchable certificate table. A scope switcher in the header allows users to filter all views by country-level domain scope without leaving the current page.

The dashboard is organised into three broad categories. Health and status views provide a quick pulse on the dataset—the overview page for summary counts, and the active-versus-expired page for a deeper breakdown by certificate state. Certificate analytics pages cover validity periods, Subject Alternative Name distributions, signature and hash algorithm analysis, geographic breakdown by issuer country, and the relationship between Certificate Authorities and the domains they cover. Security-oriented views highlight shared-key reuse groups, the ranked risk assessment for individual certificates, and the comparative CA ranking based on certificate quality metrics.

Every analytical page displays summary cards, distribution charts, and pagination tables designed for exploration. Clicking a chart segment or a table row navigates to a filtered certificate list or to the full certificate detail page, which displays the complete set of parsed fields including issuer, subject, validity, fingerprints, SANs, key usage, and ZLint results. This linking creates a consistent path from high-level trends down to individual certificate evidence.

7.6 Chapter Summary

This chapter presented the analytical framework that converts the collected certificate dataset into actionable insights. The precomputation engine materializes six analytical families on a fixed cycle, while live queries handle views that require current data. The risk assessment model provides a transparent, rule-based scoring system for certificate triage, and the CA ranking formula enables comparative evaluation of issuer-level certificate quality. The interactive dashboard ties these capabilities together through a unified interface with scope-aware filtering

and drill-down navigation. The following chapter evaluates the overall platform against its design objectives and discusses the quality of the collected dataset.

CHAPTER EIGHT

Evaluation and Limitations

8.1 Evaluation Method and Verification Results

We evaluated the platform using four methods: structural inspection, static analysis, interface traceability, and review of experimental outputs. Structural inspection verified that acquisition, dataset maintenance, analytics, application services, and presentation components use consistent data contracts. Static checks validated Python syntax, TypeScript types, and frontend lint status. These methods confirm structural soundness but do not replace full integration, load, recovery, or penetration testing.

Table 8.1: Verification results

Check	Result	Interpretation
Python syntax analysis	All first-party Python files parsed successfully	Establishes syntax validity, not runtime behavior
TypeScript compilation	Completed successfully without output generation	Establishes type consistency for the frontend project
Frontend lint analysis	Reported 22 errors and 45 warnings	Identifies remaining code-quality issues despite successful type checking
Backend runtime check	Static verification only; runtime integration remains future work	Full backend execution and endpoint behavior remain to be validated
Automated test inventory	Module-level and integration test automation remain future work	Functional, recovery, and concurrency claims require additional testing
Interface and data tracing	Major acquisition, analytics, and dashboard paths were structurally consistent	Confirms component relationships but not every runtime outcome

The verification identified several secondary contracts that require further alignment, including an auxiliary export path, a duplicate scope-switch path, and a shared-key timeline query. These paths do not alter the principal acquisition, analytical, or dashboard workflows, but they should be corrected and covered by integration tests before broader deployment.

8.2 Experimental IP-Crawler Results

The experimental IP crawler produced separate summaries for runs without Server Name Indication and with Server Name Indication. Each recorded run contains 79,184 outcomes.

Table 8.2: Experimental IP-crawler outcome summary

Outcome category	Without SNI	With SNI
SSL certificates found	1,719	1,761
No service on port 443	4,747	4,920
TCP timeouts	68,474	68,045
TLS timeouts	70	68
SSL handshake errors	428	434
Certificate parser errors	0	0
Other errors	3,746	3,956
Total recorded outcomes	79,184	79,184

The SNI-labelled run used `example.com` as a dummy hostname—a placeholder rather than the actual domain being connected to. It recorded 1,761 successful certificates, compared with 1,719 without SNI, a descriptive difference of 42 (approximately 2.4% more certificates across 79,184 addresses). Sending *any* SNI—even a mismatched dummy—yields at least as many certificates as no SNI, because some servers require an SNI value to complete the handshake. The available summary does not include repeated trials, a complete environment description, per-address results, or evidence that every other condition was held constant. The values therefore demonstrate recorded crawler behaviour but do not establish a causal effect of the specific SNI value or a population-level conclusion about Pakistan or global TLS deployment.

8.3 Security and Scalability Assessment

Security analysis distinguishes certificate measurement from controls expected in a deployed multi-user service. During collection, trust and hostname validation are disabled so that presented certificates can be observed; this activity must remain isolated from normal trusted-client behavior. The dashboard is designed for controlled operation and would require authentication, authorization, protected administrative actions, externalized secrets, restricted service exposure, and hardened environment-specific settings before broader deployment.

Currently the analysis scope is stored as shared service state. For concurrent multi-user access, each request would need to carry its own scope. If the system were deployed publicly, collection would also require an approved policy covering authorization, rate limits, opt-out handling, data retention, and responsible disclosure.

The system includes several mechanisms intended to support large datasets: concurrent acquisition, atomic task claims, batch writes, persisted checkpoints, compound indexes, materialized analytics, bounded candidate queries, pagination, optional response caching, and client-side request reuse. These mechanisms provide a scalability-oriented design, but their quantitative effect has not been established through a controlled throughput, latency, resource, or multi-user benchmark.

8.4 Limitations and Threats to Validity

The main limitations of the platform are:

1. **Configuration consistency.** All components must share the same validated dataset and service configuration to produce consistent analytical results.
2. **Dataset-update correctness.** Renewal checkpoints and temporary staging data require coordinated lifecycle management.
3. **Analytical calibration.** CA rankings, signature strength values, shared-key exposure labels, and risk scores are project-defined. They need external validation before they can be interpreted as scientific security measurements.
4. **Geographic interpretation.** Country views are based on domain suffix and configured scope, not on physical server, subject, or issuer location.
5. **Evaluation coverage.** This chapter evaluates the platform through static verification and one IP-crawler experiment. Integration, concurrency, recovery, usability, security, and performance testing remain incomplete.
6. **Deployment readiness.** Production configuration, automated deployment, dependency locking, service supervision, and comprehensive test automation still require further work.

Internal validity. Static checks cannot confirm correct behaviour under every runtime, concurrency, or interruption condition.

Construct validity. The analytical scores use project-defined rules. Shared public keys do not prove compromise, encoded expiry does not imply stale authorization, and scheduled expiration counts are not predictions.

External validity. Domain and IP inputs are bounded datasets rather than random or complete samples of global TLS deployment. The IP-crawler results are separate from the primary dashboard dataset.

Reproducibility and temporal validity. Repeatable evaluation requires frozen datasets, complete environment definitions, versioned tools, recorded commands, and dated outputs. Certificates, CT observations, network conditions, and domain ownership change over time, so analytical results represent only the state captured by a particular acquisition and refresh cycle.

CHAPTER NINE

Conclusion and Future Work

9.1 Conclusion

SSL Guardian addresses the challenge of collecting, maintaining, analyzing, and presenting large SSL/TLS certificate datasets through one integrated platform. The developed system combines concurrent endpoint crawling, structured X.509 parsing, CT-assisted renewal and new-domain discovery, persistent certificate storage, global and country-level analytics, application services, and an interactive dashboard.

The acquisition framework uses persistent task states, atomic worker claims, concurrent processing, retries, stale-task recovery, certificate parsing, scope enrichment, and leaf classification. A separate IP-based crawler extends the acquisition work to address-based measurement while remaining isolated from the primary domain-based analytical dataset.

The CT pipeline supports continuous dataset development by observing CT-derived domain sets, selecting known-domain renewal candidates, discovering new domains, crawling their current endpoints, and incorporating successful results. Analytical materialization follows completed update cycles, allowing the dashboard to reflect newly acquired certificate information with minimal operational delay while preserving a clear boundary from strict event-by-event real-time processing.

The analytical platform covers validity, issuer and Certificate Authority activity, signature and hash algorithms, key properties, SANs, public-key reuse, geographic scope, trends, and security-oriented risk indicators. Certificate Authority ranking and certificate-risk scoring provide transparent comparative views, while the dashboard connects aggregate visualizations to filtered lists and detailed certificate records. These scores and labels are application-defined analytical aids and should not be interpreted as proof of compromise or externally certified security ratings.

The project demonstrates a broad systems-engineering contribution to certificate intelligence. Its principal strengths are the integration of acquisition, CT-driven dataset maintenance, analytical processing, and interactive exploration. The evaluation also identifies limitations in production hardening, test automation, request isolation, benchmark coverage, and analytical calibration. These boundaries define the present scope and provide a clear direction for continued development.

9.2 Future Work

Future engineering work should strengthen the platform through systematic unit, integration, recovery, service-contract, and interface testing, supported by reproducible datasets and explicit

dependency versioning. Unified configuration, externalized secrets, environment-specific settings, authentication, role-based authorization, request-level scope isolation, automated deployment, and operational monitoring would further support secure and reproducible use beyond a locally operated research environment.

Larger deployments would benefit from distributed crawling, durable task coordination, resource-aware scheduling, backpressure, and idempotent processing. Incremental or distributed analytics could reduce repeated full-dataset computation as the certificate dataset grows. Controlled benchmarks should accompany these changes to evaluate acquisition throughput, update latency, query responsiveness, and resource consumption across larger datasets and analytical scopes.

Certificate intelligence may be extended through certificate-chain and trust path analysis, deeper examination of certificate policies and name constraints, and active OCSP or CRL status assessment. Retaining historical certificate versions would enable lifecycle, renewal, key-rotation, issuer-migration, and stale-deployment studies. Certificate Authority evaluation could consequently incorporate transparent longitudinal measures, while ownership-aware analysis could distinguish legitimate shared infrastructure from concerning public-key reuse [6].

Certificate Transparency processing could progress from scheduled new-domain and renewal updates toward continuous, long-term ecosystem observation. Durable per-log progress, event histories, and incremental correlation with acquired certificates could improve renewal detection and reveal unusual changes in issuers, public keys, SAN coverage, validity profiles, or issuance frequency. Such anomalies should remain explainable investigative indicators.

Machine learning and artificial intelligence offer a complementary research direction when sufficient history and reliable labels become available. Certificate attributes, CT observations, issuer behaviour, renewal history, cryptographic properties, quality findings, and analytical indicators could support anomaly detection, risk estimation, suspicious-certificate prioritization, and analyst triage. Models would require calibration, interpretability, bias assessment, and concept-drift monitoring; they should not be claimed to detect hacked websites directly or predict attacks.

Further extensions may integrate governed threat intelligence, including trust-store, passive-DNS, reputation, and known malicious infrastructure data, with clear provenance and licensing controls. IPv6 measurement, broader deployment environments, and richer longitudinal and comparative visualizations would expand research utility. Collectively, these directions reinforce the platform as a foundation for large-scale SSL/TLS certificate intelligence, cybersecurity research, and advanced analytical systems.

REFERENCES

- [1] E. Rescorla, “The transport layer security (TLS) protocol version 1.3,” RFC Editor, RFC 8446, Aug. 2018. [Online]. Available: <https://www.rfc-editor.org/info/rfc8446>
- [2] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. T. Polk, “Internet x.509 public key infrastructure certificate and certificate revocation list (CRL) profile,” RFC Editor, RFC 5280, May 2008. [Online]. Available: <https://www.rfc-editor.org/info/rfc5280>
- [3] B. Laurie, E. Messeri, and R. Stradling, “Certificate transparency version 2.0,” RFC Editor, RFC 9162, Dec. 2021. [Online]. Available: <https://www.rfc-editor.org/info/rfc9162>
- [4] F. Nezhadian, E. Branca, A. Barzolevskaia, A. Natadze, and N. Stakhanova, “Measuring and characterizing propagation of reuse RSA certificates and keys across PKI ecosystem,” *IEEE/ACM Transactions on Networking*, vol. 33, no. 2, pp. 595–611, 2025.
- [5] Z. Ma, A. Faulkenberry, T. Papastergiou, Z. Durumeric, M. D. Bailey, A. D. Keromytis, F. Monroe, and M. Antonakakis, “Stale TLS certificates: Investigating precarious third-party access to valid TLS keys,” in *Proceedings of the 2023 ACM Internet Measurement Conference*, 2023, pp. 222–235.
- [6] K. Junaid and M. U. Janjua, “Analyzing shared keys in X.509 certificates with domain ownership,” *International Journal of Information Security*, vol. 24, 2025, article 106.
- [7] Z. Durumeric, J. Kasten, M. Bailey, and J. A. Halderman, “Analysis of the HTTPS certificate ecosystem,” in *Proceedings of the 2013 ACM Internet Measurement Conference*, 2013, pp. 291–303.
- [8] D. Kumar, Z. Wang, M. Hyder, J. Dickinson, G. Beck, D. Adrian, J. Mason, Z. Durumeric, J. A. Halderman, and M. Bailey, “Tracking certificate misissuance in the wild,” in *2018 IEEE Symposium on Security and Privacy*, 2018, pp. 785–798.
- [9] crt.sh, “Certificate search and certificate transparency log monitor,” <https://crt.sh/>, accessed: Jul. 16, 2026.
- [10] Z. Durumeric, D. Adrian, A. Mirian, M. Bailey, and J. A. Halderman, “A search engine backed by internet-wide scanning,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 542–553.

- [11] Censys, “Certificates,” <https://docs.censys.com/docs/ls-certificates>, accessed: Jul. 16, 2026.
- [12] Shodan, “What Is Shodan?” <https://help.shodan.io/the-basics/what-is-shodan>, accessed: Jul. 16, 2026.
- [13] “Shodan — Certificate Transparency Log,” <https://book.shodan.io/developer-apis/certificate-transparency/>, accessed: Jul. 16, 2026.